

RL в рекомендациях

ШАД, Рекомендательные системы 2025/26

Руслан Кулиев

ML-разработчик

Яндекс Лавка, группа рекомендательных и поисковых сервисов

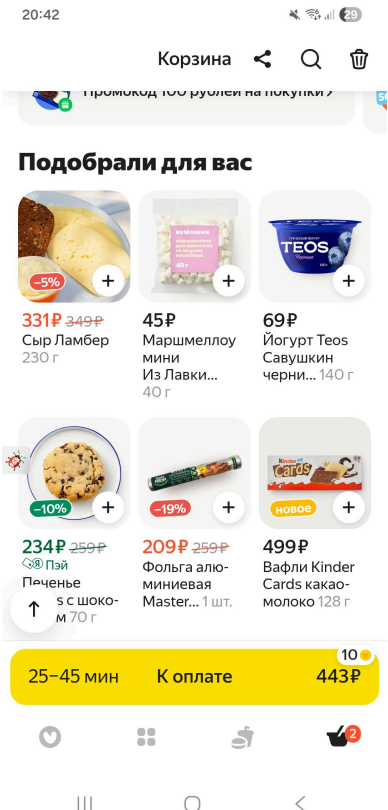
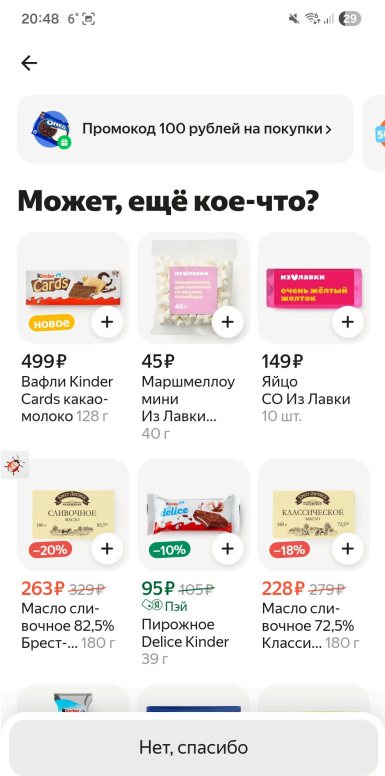
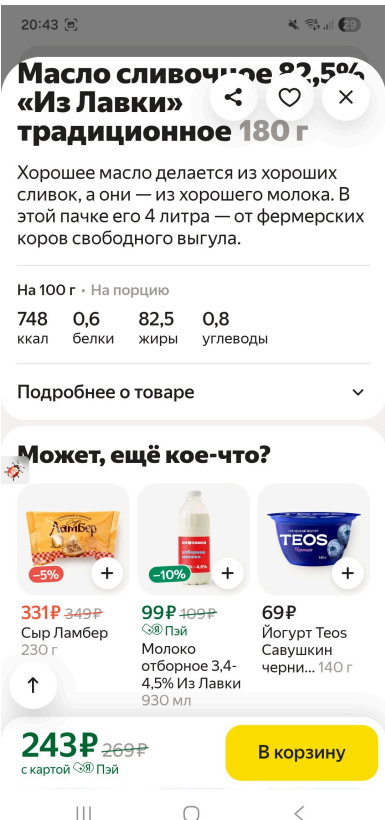
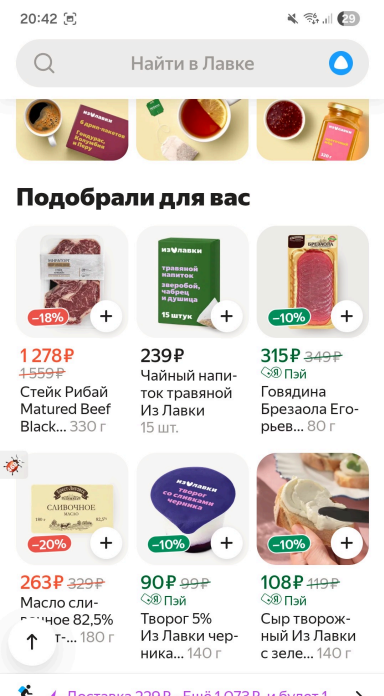
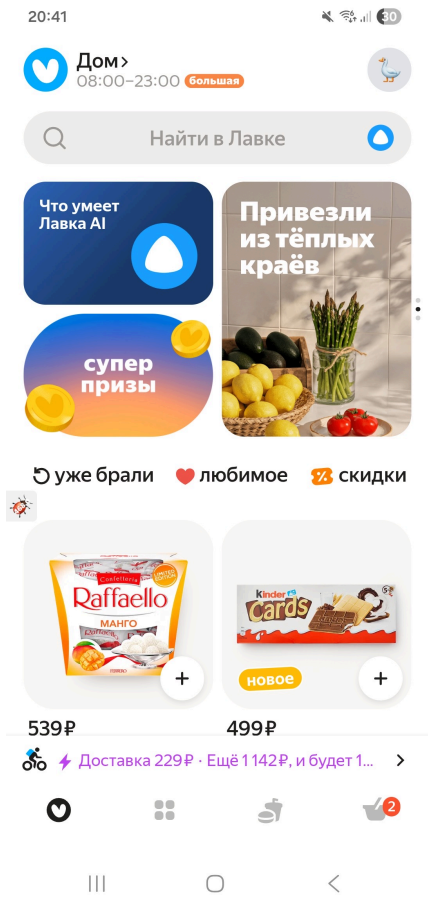
План занятия

1. Real world application
2. От ранжирования к бандитам
3. От бандитов к RL
4. O Slate Recommendation
5. Off Policy

Real world application

Часть 1

Яндекс Лавка



Яндекс Лавка

- E-com (e-grocery)
- Сильный интент
- Повторяемость покупок
- Множество активных рекомендательных поверхностей
- Конкуренция поверхностей и каннибализация

Каннибализация

- Активность пользователя просто перераспределяется
- Поверхности конкурируют за внимание пользователя
- Поверхности конкурируют за айтемы

От рекомендаций к бандитам

Часть 2

Какую задачу решаем в ранжировании?

Простой случай – лента на главной

Хотим, чтобы пользователь купил товар, который мы ему покажем

Supervised prediction – хотим получить какую-то оценку релевантности

- $p(\textit{purchase} \mid \textit{user}, \textit{item}, \textit{context})$
- $\textit{score}(\textit{user}, \textit{item}, \textit{context})$

Но учимся мы на $p(\textit{purchase} \mid \dots, \textit{view})$ - т.е. весь фидбэк, что у нас есть – результат текущей логирующей политики

- Feedback loop, selection bias и все такие прелести
- $p(\textit{purchase}) = p(\textit{purchase} \mid \textit{view}) \cdot p(\textit{view})$
- Можем учить только $p(\textit{purchase} \mid \textit{view})$

Как бороться с feedback loop

А зачем?

- Система будет очень плохо работать с новыми объектами
- Выучиваемся на «что работает», а не «что действительно релевантно»

Можно добавить стохастичность или inductive bias

- Сэмплирование по вероятности
- Квоты на группы товаров
- Подмешивать рандомные товары

Но это скорее инженерные трюки и они требуют аккуратной ручной настройки

А можно сделать эту систему более умной и самой

Бандиты

Проблема **exploration vs exploitation**

- Есть частичный фидбэк и много неопределенности
- Надо балансировать между релевантностью и потребностью в знании

Бандиты – способ принимать решения в рамках такой неопределенности
Они своего рода надстройка над ранжирующей моделью – ранкер дает представление о релевантности и базовый слейт, но бандит, в зависимости от онлайн-фидбэка и суперпозиции релевантности и неопределенности (потребности в новом знании), может слегка переранжировать список или вставлять новые элементы

Модели

Epsilon-greedy

- С некоторой вероятностью просто заменяем товар из exploration-сета
- Де факто подбрасывание монетки
- Больше инженерный трюк

Модели

UCB

- Upper Confidence Bound
- $score = ctr + \alpha \sqrt{\frac{2 \log t}{n}}$
 - t – общее количество показов, n – показы данного айтема
- Выводится из неравенства Хёфдинга
 - Есть гарантии, что истинный CTR лежит ниже полученного $score$
- По умолчанию ctr берем исторический, но на практике это $p(ctr)$ из нашей модели
 - Из минусов – теоретические гарантии рушатся, но на практике работает хорошо
- Из минусов – алгоритм детерминирован

Модели

Thompson sampling

- Байесовские трюки
- Сэмплируем скор из бета-распределения, $score \sim Beta(\alpha, \beta)$
- α – число успехов (+1), β – число неудач (+1)
- Обновляем по мере получения фидбэка
- У каждого товара есть ненулевая вероятность быть показанным – далее посмотрим, почему это *критически* важно

Из интересных трюков

- Можно инициироваться не обязательно нулями, а какими-то средними значениями, например, в срезе категории – так сходимость произойдет быстрее
- В DL-моделях можно сделать простую аппроксимацию – можно включить dropout на инференсе

Бандиты

Т.о. бандиты помогают корректировать логирующую политику в рантайме. Но трюк концептуально довольно общий, и может применяться для разных задач и в разных местах пайплайна:

- Подмешивание новые для пользователя товары
- Прогрев холодные товары
- Буст категорий товаров (напр. СТМ)
 - В частности, рекламные механизмы – это тоже во многом бандитные механики
- Баланс кандидатогенераторов
- А/В тесты

От бандитов к RL

Часть 3

RL

Как формулируется абстрактная rl-задача:

- Есть среда как сущность, с которой мы пытаемся взаимодействовать, и пусть она даже статична
- Есть агент, у которого есть состояние и который может взаимодействовать со средой
- Есть набор действий, который может совершить агент
- После фидбэка от среды мы переходим в другой стейт
- И есть некоторая политика, которая определяет, как мы будем выбирать оптимальные действия, чтобы максимизировать некоторую *будущую* награду

Рекомендательная система в терминах RL

- Среда – это пользователь
- Стейт – это контекст (история покупок, корзина, etc.)
- Агент – это наша рекомендательная система
- Действия – это айтемы, которые мы покажем пользователю

Итого, мы хотим получить хорошую политику, т.е. научиться показывать пользователю такую хорошую последовательность айтемов (реактивно реагируя на фидбэк), чтобы максимизировать долгосрочную пользу

Вернемся к бандитам

- Бандит – тривиальный пример rl-системы
- Если ранкер сам по себе оптимизируем immediate profit, т.е. оценивает, например, $p(\text{purchase} \mid \text{item}, \text{user}, \text{context})$, то бандит оптимизирует $\text{score} = p(\text{purchase} \mid \text{item}, \text{user}, \text{context}) + \text{uncertainty}$
- NB: здесь нет долгосрочного профита, score как аналог reward вычисляется на одном шаге

Вернемся к Лавке

У нас есть следующие принципиальные таргеты для оптимизации

- $p(\text{purchase} \mid \text{item}, \text{user}, \text{context})$ – immediate reward своего рода. Мы хотим, чтобы каждый сниппет был хорош и пользователь был готов его купить
- $E_{\text{session}}[\text{order_price}]$ – мы уже выходим на уровень сессии: хотим максимизировать сумму в корзине
- $E_{\text{user}}[LTV]$ – святой Грааль, то, чего хотел бы бизнес и чего вообще все хотели бы. Выстраиваем рекомендации так, чтобы пользователь к нам возвращался и приносил прибыль

Оптимизация на уровне сессии

Проблемы

- Траектории юзера крайне вариативны
- Каннибализация
 - В частности, $p(view) = p(view | surface) * p(surface)$
 - Если постараться, можно даже что-то вроде аукциона выписать

Пусть у нас есть единая модель и мы максимизируем $E_{session}[order_price]$

Мы смотрим на траекторию пользователя на протяжении всей сессии

- $r_0 = 0$
- $R_{session} = \sum_i Price_i$ по купленным товарам

Показ Яндекс Станции с высокой стоимостью – интересно, но почти наверняка бесполезно

Показ какой-нибудь свеклы может привести к корзине с ингредиентами для борща

Модель должна сама выучить все эти взаимоотношения

REINFORCE

- $\pi(a|s)$ - наша рекомендательная система (политика)
- Ну и пусть мы хотим т.о. в единой модели оптимизировать что-то долгосрочное
- $J(\theta) = E_{\tau \sim \pi_\theta}[R(\tau)]$ – матожидание вознаграждения
- А чтобы на это учиться, возьмем градиент:
- $\nabla_\theta J(\theta) = E[\sum_t (G_t - b(s_t)) \nabla_\theta \log \pi_\theta(a_t|s_t)]$
 - $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$
 - $b(s)$ – бейзлайн
 - Какая-то эвристика
 - Оценка другой модели (т.н. actor-critic система)
 - Оценка этой же модели (модные модификации пункта выше)

Проблемы ванильного REINFORCE

- Sparse reward
 - Пространство действий огромно – десятки тысяч айтемов в Лавке, миллионы – в Музыке
- Высокая дисперсия
 - Берем всю сумму наград до конца сессии
 - Но многие вещи по типу интента юзера от модели не зависят, вносят много шума и замедляют сходимость
- Проблема распределения наград
- Item-based
 - В действительности юзеры не видят айтемы one-by-one, по крайней мере в Лавке
 - Мы формируем выдачу слейтами
- On policy
 - Должен обучаться на той же политике
 - Просто не можем себе позволить в рантайме

O Slate recommendation

Часть 4

Слейты

Почему они вообще возникают

- Естественная природа вещей
- Инфраструктурные ограничения

Как это влияет на постановку задачи

- Action - это уже не item, а slate целиком
- В слейте айтемы влияют друг на друга явно
 - Заменители
 - Дополнители
 - Positional bias
- Immediate reward будет неоптимален – и ранкер, и классические бандиты, даже если будут показывать хорошее качество, вряд ли будут близки к оптимальным

Слейты

Пусть $R(a_i)$ - награда за показ одного айтема, тогда $R(A)$ - награда за слейт целиком

- $R(A) \neq \sum_i R(a_i)$

Концептуально REINFORCE все еще работает:

- $\nabla_{\theta} J(\theta) = E[\sum_t (R(A) - b(s)) \nabla_{\theta} \log \pi_{\theta}(A|s_t)]$

Загадки Жака Фреско:

- $\pi_{\theta}(A|s_t)$ - комбинаторный взрыв
- Как распределить награду по слейту

Генерация слейтов

Надо как-то разложить $\pi_\theta(A|s_t)$, чтобы не получать триллионы вариантов

- Авторегрессионные модели
 - $\pi_\theta(A|s_t) = \prod_k \pi_\theta(a_k|a_1, \dots, a_{k-1})$
 - Генерируем лучших кандидатов, обуславливаемся на них при последующей генерации
 - Мечта для decoder
- Plackett-Luce
 - $\pi_\theta(A|s_t) = \prod_k \frac{\exp(z_{a_k})}{\sum_{U_k} \exp(z)}$
 - U_k — множество ещё не выбранных кандидатов на шаге k
 - Большая дисперсия градиента, т.к. не учитывается, как перестановка влияет на награду

Распределение награды

Если давать награду всему слейту – дисперсия будет высокой, а обучение шумным

Делают position wise decomposition

- Пересчитывают награду за слейт, чтобы учитывать только последующие позитивы
- $\nabla_{\theta} J(\theta) = E[\sum_t (G_t - b_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t, \dots)]$

Off policy

Часть 5

Проблема

Идеальный мир

- On policy – выкатываем новую логирующую политику, считаем награду, учимся, repeat & profit
- Практически недостижимо

Partial feedback

- У нас есть фидбэк только по тем айтемам, что мы показали пользователю
- И совершенно нет представления, как бы пользователь отреагировал на:
 - Принципиально новый товар
 - Заменитель
 - Даже на иной порядок выдачи
- У нас есть только траектории, собранные старой логирующей политикой
- Support mismatch
 - Оффлайн-оценка на совершенно непоказанных данных в принципе невозможна

Inverse propensity scoring

У нас есть траектории старой логирующей политики $\mu(A|x)$

У нас появилась новая политика $\pi(A|x)$

Можно сделать трюк: предположить, насколько лучше новая политика, перевзвешивая награду

- $$J_{IPS} = \frac{1}{N} \sum_i \frac{\pi(A_i|x)}{\mu(A_i|x)} R(A_i)$$

Несмещенность полученной оценки:

$$E_{A \sim \mu} \left[\frac{\pi(A)}{\mu(A)} R(A) \right] = \sum_A \mu(A) \frac{\pi(A)}{\mu(A)} R(A) = \sum_A \pi(A) R(A) = E_{A \sim \pi} [R(A)]$$

Off policy оценка с помощью IPS

- Kish's approximation: $ESS = \frac{(\sum w)^2}{\sum w^2}$, где w – веса из IPS
- Чем ниже ESS, тем меньше реальных данных поддерживает нашу оффлайн-оценку. При $ESS < \text{порога}$ оффлайн-метрикам доверять нельзя

Суровая реальность

- Логирующая политика должна хотя бы иногда показывать комбинации товаров, которые хочет показывать новый алгоритм
 - Для IPS требуется, чтобы $\mu(A) > 0$ был везде, где $\pi(A) > 0$
 - В силу этого offline REINFORCE будет штрафовать инновационные траектории, какими бы хорошими они ни были
- На слейтах и длинных траекториях вероятность раскладывается в произведение вероятностей -> градиенты будут взрываться
 - Если на каждом из 10 слотов дробь $\pi/\mu = 3$, то вес слейта = $3^{10} = 59\,000$
- Случаи, когда старая политика **случайно** оказалась хороша (вернула товар с крайне маленьким шансом на показ), то веса тоже взорвутся и мы моментально переобучимся

Стабилизируем IPS

- Weight clipping
 - $w_i^{clip} = \min\left(\frac{\pi(A_i|x_i)}{\mu(A_i|x_i)}, C\right)$
 - Снижаем дисперсию и стабилизируем обучение, но вносим систематическое смещение
- Self normalization
 - $J_{SNIPS} = \frac{\sum_{i=1}^N w_i \cdot R(A_i)}{\sum_{i=1}^N w_i}$
- Doubly robust
 - $J_{DR} = \frac{1}{N} \sum_{i=1}^N \left[\hat{R}(x_i, A_i^\pi) + \frac{\pi(A_i|x_i)}{\mu(A_i|x_i)} (R(A_i) - \hat{R}(x_i, A_i)) \right]$
 - Параллельно с IPS обучаем отдельную модель $\hat{R}(x, A)$, которая предсказывает награду за слейт напрямую
 - Оценка остаётся несмещённой, если выполняется хотя бы одно из двух условий — либо propensities μ корректны, либо reward-модель $\hat{R}$ точна

Итого

Итого

- Классические системы решают задачу ранжирования жадно
- Это неоптимально с точки зрения бизнеса
 - Мы хотим оптимизировать другие метрики
- Это неоптимально с точки зрения инженерии
 - Мы загоняем себя в feedback loop и плохо понимаем интересы пользователя
- Бандиты – способ вырваться из порочного круга, дать модели больше воздуха и за минимальный бюджет позволить системе работать лучше
 - Thompson sampling – топ за свои деньги
- RL – сказка, мечта, которая должна решить все проблемы, но не заводится тривиально
- REINFORCE – довольно простой вывод, как учить модели на долгосрочную награду
- Top-K – более естественная постановка задачи, которая добавляет трудностей в обучении
- Off policy – суровая реальность, которая вынуждает корректировать REINFORCE

Спасибо!

Руслан Кулиев
*ML-разработчик
Группа рекомендательных и поисковых сервисов
Яндекс Лавка*

Санкт-Петербург 2026